Prikshit Kumar Angra, Lovely Professional University

Unit 02: Constructors and Destructors

CONTENTS

Objectives

After studying this unit, you will be able to:

- Recognize the need for constructor and destructors
- Describe the copy constructor
- Explain the dynamic constructor
- Discuss the destructors
- Explain the constructor and destructors with static members

Introduction

When an object is created all the members of the object are allocated memory spaces. Each object has its individual copy of member variables. However, the data members are not initialized automatically. If left uninitialized these members contain garbage values. Therefore, it is important that the data members are initialized to meaningful values at the time of object creation. Conventional methods of initializing data members have lot of limitations. In this unit you will learn alternative and more elegant ways initializing data members to initial values.

When a C++ program runs it invariably creates certain objects in the memory and when the program exits the objects must be destroyed so that the memory could be reclaimed for further use.

C++ provides mechanisms to cater to the above two necessary activities through constructors and destructors methods.

2.1 Constructor and Destructor

Constructor

Constructors are special class functions which performs initialization of every object. The Compiler calls the Constructor whenever an object is created. Constructors initialize values to object members after storage is allocated to the object. Whereas Destructor on the other hand is used to destroy the class object.

The default constructor method is called automatically at the time of creation of an object and does nothing more than initializing the data variables of the object to valid initial values.


Notes: Constructor has the same name as the class. Constructor is public in the class. Constructor does not have any return type.

While writing a constructor function the following points must be kept in mind:

1. The name of constructor method must be the same as the class name in which it is defined.
2. A constructor method must be a public method.
3. Constructor method does not return any value.
4. A constructor method may or may not have parameters.

Let us examine a few classes for illustration purpose. The class abc as defined below does not have user defined constructor method.

```
classabc
{
}
main()
{
}
intx,y;
abcmyabc;
…;
```

The main function above an object named myabc has been created which belongs to abc class defined above. Since class abc does not have any constructor method, the default constructor method of C++ will be called which will initialize the member variables as:

myabc.x=0 and myabc.y=0.

Let us now redefine myabc class and incorporate an explicit constructor method as shown

below:

classabc

Observed that myabc class has now a constructor defined to except two parameters of integer type. We can now create an object of myabc class passing two integer values for its construction, as listed below:

```
main()
{
```

```
abcmyabc(100,200);
———;
}
```

In the main function myabc object is created value 100 is stored in data variable x and 200 is stored in data variable y. There is another way of creating an object as shown below.

```
main()
{
myabc=abc(100,200);
———;
}
```

Both the syntaxes for creating the class are identical in effect. The choice is left to the programmer. There are other possibilities as well. Consider the following class differentials:

```
classabc
{
intx,y; public:
abc();
}
abc::abc()
{
x=100; y=200;
}
```

In this class constructor has been defined to have no parameter. When an object of this class is created the programmer does not have to pass any parameter and yet the data variables x,y are initialized to 100 and 200 respectively.

Finally, look at the class differentials as given below:

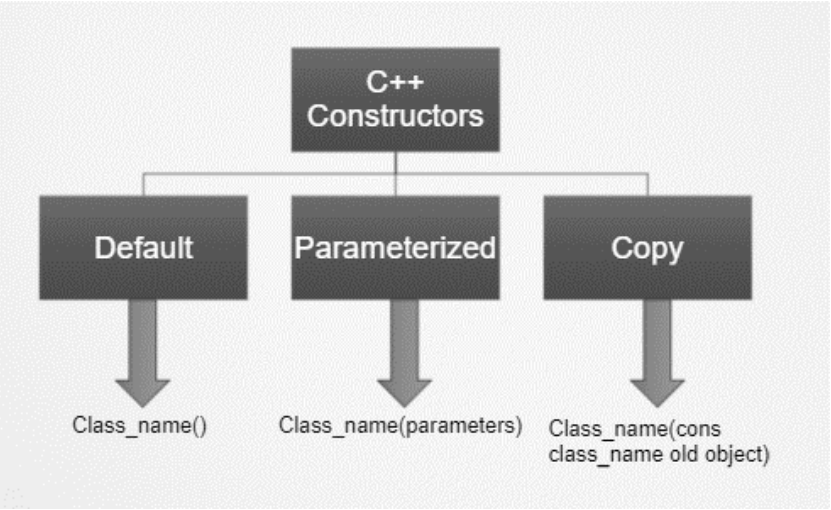
```
classabc

{

intx,y; public:
abc();

abc(int);

abc(int, int);

}

abc::abc()

{
```

```
x=100; y=200;
}

abc::abc(int a)

{

x=a; y=200;
}

abc::abc(int a)

{

x=100;

y=a;

}
```

Class myabc has three constructors having no parameter, one parameter and two parameters respectively. When an object to this class is created depending on number of parameters one of these constructors is selected and is automatically executed.

Types of constructor

1. Default Constructor
2. Parameterized Constructor
3. Copy Constructor



Destructor

Destructors are typically used to de-allocate memory. Also, they are used to clean up for objects and class members when the object gets terminated.

When should you define your own destructor function? In many cases you do not need a destructor function. However, if your class created dynamic objects, then you need to define your own destructor in which you will delete the dynamic objects. This is because dynamic objects cannot be deleted on their own. So, when the object is destroyed, the dynamic objects are deleted by the destructor function you define.

A destructor function has the same name as the class, and does not have a returned value. However you must precede the destructor with the tilde sign, which is ~ .

The following code illustrates the use of a destructor against dynamic objects:

```
#include <iostream> using namespace std;

class Calculator
{
public:
int *num1;
int *num2;

Calculator(int ident1, int ident2)
{
num1 = new int;
num2 = new int;
*num1 =ident1;
*num2 =ident2;
}
~Calculator()
{
delete num1;
delete num2;
}
int add(){
int sum=*num1+*num2;
return sum;
}
};
int main()
{
Calculator myObject(20,20);
int result = myObject.add();
cout<< result;
```

```
return 0;
}
```

The destructor function is automatically called, without you knowing, when the program no longer needs the object. If you defined a destructor function as in the above code, it will be executed. If you did not define a destructor function, C++ supplies you one, which the program uses unknown to you. However, this default destructor will not destroy dynamic objects.

Notes:-An object is destroyed as it goes out of scope.

Lab Exercise: Program to see how Constructor and Destructor are called.

```
class A
{
 // constructor
A()
 {
cout<< "Constructor called";
 }
// destructor
 ~A()
 {
cout<< "Destructor called";
 }
};
int main()
{
 A obj1; // Constructor Called
int x = 1
if(x)
 {
 A obj2; // Constructor Called
 } // Destructor Called for obj2
} // Destructor called for obj1
```

Output:-

```
Constructor called
Constructor called
Destructor called
Destructor called
```

2.2 Difference between Constructor and Destructor in C++

Constructors	Destructors
The constructor initializes the class and allots the memory to an object.	If the object is no longer required, then destructors demolish the objects.
When the object is created, a constructor is called automatically.	When the program gets terminated, the destructor is called automatically.
It receives arguments.	It does not receive any argument.
A constructor allows an object to initialize some of its value before it is used.	A destructor allows an object to execute some code at the time of its destruction.
It can be overloaded.	It cannot be overloaded.
When it comes to constructors, there can be various constructors in a class.	When it comes to destructors, there is constantly a single destructor in the class.
They are often called in successive order.	They are often called in reverse order of constructor.

2.3 Copy Constructor

A copy constructor method allows an object to be initialized with another object of the same class. It implies that the values stored in data members of an existing object can be copied into the data variables of the object being constructed, provided the objects belong to the same class. A copy constructor has a single parameter of reference type that refers to the class itself as shown below:

```
abc::abc(abc& a)


{
x=a.x;
y=a.y;
}
```

Suppose we create an object myabc1 with two integer parameters as shown below:

```
abc myabc1(1,2);
```

Having created myabc1, we can create another object of abc type, say myabc2 from myabc1, as shown below:

```
myabc2=abc(& myabc1);
```

The data values of myabc1 will be copied into the corresponding data variables of object myabc2. Another way of activating copy constructor is through assignment operator. Copy constructors come into play when an object is assigned another object of the same type, as shown below:

```
abc myabc1(1,2);
abc myabc2;
myabc2=myabc1;
```

Actually assignment operator(=) has been overloaded in C++ so that copy constructor is invoked

whenever an object is assigned another object of the same type.

Did you know?

What is the difference between the copy constructor and the assignment operator?

(a) If a new object has to be created before the copying can occur, the copy constructor isused.

(b) Ifanewobjectdoesnothavetobecreatedbeforethecopyingcanoccur,theassignment operator isused.

2.4 Dynamic Constructor

- Dynamic constructor is used to allocate the memory to the objects at the run time.
- Memory is allocated at run time with the help of 'new' operator.
- By using this constructor, we can dynamically initialize the objects.

Example:-

```
#include <iostream.h>
#include <conio.h>
classdyncons
{
int * p;
public:
dyncons()
 {
  p=new int;
  *p=100;
 }
dyncons(int v)
 {
  p=new int;
  *p=v;
 }
int dis()
 {
return(*p);
 }
};
int main()
{
clrscr();
dyncons o, o1(90);
```

```
cout<<"The value of object o's p is:";
cout<<o.dis();
cout<<"\nThe value of object 01's p is:"<<o1.dis();
return 0;
}
```

Output:

The value of object o's p is:100

The value of object 01's p is:90

2.5 Parameterized Constructors

If it is necessary to initialize the various data elements of different objects with different values when they are created. C++ permits us to achieve this objective by passing arguments to the constructor function when the objects are created. The constructors that can take arguments are called 'Parameterized constructors.' The definition and declaration are as follows:

```
classdist
{
int m, cm; public:
dist(int x, int y);
};
dist::dist(int x, int y)
{
m = x;     n = y ;
}
main()
{
dist d(4,2);
d. show ();
}
```

2.6 Constructors with Default Arguments

This method is used to initialize object with user defined parameters at the time of creation.

Consider the following Program that calculates simple interest. It declares a class interest representing principal, rate and year. The constructor function initializes the objects with principal and number of years. If rate of interest is not passed as an argument to it the Simple Interest is calculated taking the default value of rate of interest.

```
#include <iostream.h>
class interest
{
int principal, rate, year;
float amount;
```

```
public:
interest (int p, int n, int r = 10);
voidcal (void);
};
interest::interest (int p, int n, int r = 10)
{
principal = p;
year = n;
rate = r;
};
void interest::cal (void)
{
cout<< "Principal" <<principal;
cout<< "\ Rate" <<rate;
cout<< "\  Year"  <<year; amount = (float) (p*n*r)/100;
cout<< "\Amount" <<amount;
};
main ( )
{
interest i1(1000,2);
interest i2(1000, 2,15);
il.cal();
i2.cal();
}
```

Two objects are created in main function

```
interest i1(1000,2);
interest i2(1000, 2,15);
```

The data members principal and year of object i1 are initialized to 1000 and 2 respectively at the time when object i1 is created. The data member rate takes the default value 10 whereas when the object 12 is created, principal, year and rate are initialized to 1000, 2 and 15 respectively.

It is necessary to distinguish between the defaults

constructor::construct();

and default argument constructor

construct::construct(int = 0)

The default argument constructor can be called with one or no arguments. When it is invoked with no arguments it becomes a default constructor. But when both these forms are used in a class, it causes ambiguity for a declaration like construct C1;

The ambiguity is whether to invoke construct: : construct ( ) or construct: : construct (int=O)

Lab Exercise

```
//# Program – Default constructor
#include<iostream>
using namespace std;
class constructor{
private:
intx,y;
public:
constructor(){
   x=10;
   y=90;
cout<<"Sum of x and y is :"<<x+y;
   }
   };
int main(){
constructor c;
return 0;
   }
```

Output

```
Sum of x and y is :100
Process returned 0 (0x0)   execution time : 0.014 s
Press any key to continue.
```

Lab Exercise

// Program –Parameterizedconstructor

```
#include<iostream>
using namespace std;
class constructor{
private:
intx,y;
public:
constructor(inta,int b){
   x=a;
y=b;
cout<<"Sum of x and y is :"<<x+y;
   }
   };
int main(){
```

```
constructor c(15,52);
return 0;
  }
```

Output

```
Sum of x and y is :67
Process returned 0 (0x0)   execution time : 0.313 s
Press any key to continue.
```

Lab Exercise

// Program – Copy Constructor

```
#include<iostream>
using namespace std;
classcopyconstructor
{
private:
int x, y;
public:
copyconstructor(int x1, int y1)
  {
    x = x1;
    y = y1;
  }
copyconstructor (constcopyconstructor&sam)
  {
    x = sam.x;
    y = sam.y;
  }

void display()
  {
cout<<x<<" "<<y<<endl;
  }
};
int main()
{
copyconstructor obj1(10, 15);
copyconstructor obj2 = obj1;
cout<<"Constructor : ";
```

```
obj1.display();
cout<<"Copy constructor : ";
obj2.display();
return 0;
}
```

Output

```
Constructor : 10 15
Copy constructor : 10 15

Process returned 0 (0x0)   execution time : 0.105 s
Press any key to continue.
```

Lab Exercise

// Program – Dynamic Constructor

```
#include <iostream>
using namespace std;
class demo {
int* p;
public:
demo()
   {
      p = new int;
       *p = 500;
   }
void display()
{
cout<<"Dynamic constructor"<<endl;
cout<< *p <<endl;
   }
};
int main()
{   demoobj =  demo();
obj.display();
return 0;
}
demo()
   {
      p = new int;
       *p = 500;
```

```
  }


demo(int a)
  {
    p = new int;
     *p = a;
  }
```

Output

```
Dynamic constructor
500

Process returned 0 (0x0)   execution time : 0.110 s
Press any key to continue.
```

2.7 Constructor overloading

- A class can have multiple constructors that assign the fields in different ways.
- Overloaded constructors have the same name as class name but the different number of arguments.
- A constructor is called depending upon the number and type of arguments passed.
- While creating the object, arguments must be passed to let compiler know, which constructor needs to be called.

```
//Program
#include<iostream>
using namespace std;
class demo{
intx,y;
public:
demo(){
x=10;
cout<<"X is ="<<x<<endl;

}
demo(inta,int b){
   x=a;
   y=b;
cout<<"Sum of x and y is"<<x+y;
}
};
```

```
main(){
demo d1,d2(90,59);
}
```

Output

2.8 Destructors

Constructors create an object, allocate memory space to the data members and initialize the data members to appropriate values; at the time of object creation. Another member method called destructor does just the opposite when the program creating an object exits, thereby freeing the memory.

A destructive method has the following characteristics:

1. Name of the destructor method is the same as the name of the class preceded by a tilde (~).
2. The destructor method does not take any argument.
3. It does not return any value.

The following codes snippet shows the class student with the destructor method;

Example

```
#include <iostream>
using namespace std;
class student
 {
public:
student()
     {
cout<<"Constructor Invoked"<<endl;
     }
     ~student()
     {
cout<<"Destructor Invoked"<<endl;
     }
};
int main()
{
```

```
student s1;
return 0;
}
```

Summary

- A constructor is a member function of a class, having the same name as its class and which is called automatically each time an object of that class it created.
- It is used for initializing the member variables with desired initial values. A variable (including structure and array type) in C++ may be initialized with a value at the time of its declaration.
- The responsibility of initialization may be shifted, however, to the compiler by including a member function called constructor.
- A class constructor, if defined, is called whenever a program created an object of that class. Constructors are public member unless otherwise there is a good reason against.
- A constructor may take argument (s). A constructor may take no argument(s) is known as default constructor.
- A constructor may also have parameter (s) or argument (s), which can be provided at the time of creating an object of that class.
- C++ classes are derived data types and so they have constructor (s). Copy constructor is called whenever an instance of same type is assigned to another instance of the same class.
- If a constructor is called with a smaller number of arguments than required an error occurs. Every time an object is created its constructor is invoked.
- The functions that is automatically called when an object is no more required is known as destructor. It is also a member function very much like constructors but with an opposite intent.

Keywords

Constructor: A member function having the same name as its class and that initializes class objects with legal initial values.

Copy Constructor: A constructor that initializes an object with the data values of another object.

Default Constructor: A constructor that takes no arguments.

Destructor: A member function having the same name as its class but preceded by ~ sign and that de initializes an object before it goes out of scope.

Self-Assessment

1. Which of the followings is/are automatically added to every class, if we do not write our own?

A. Copy Constructor
B. Assignment Operator
C. A constructor without any parameter
D. All of the above

2. What will be output of following program?

```
#include<iostream>
using namespace std;
class Point {
   Point() { cout<< "Constructor called"; }
};
int main()
{
  Point t1;
   return 0;
}
```

A. Compiler Error
B. Runtime Error
C. Constructor called
D. None of Above

3. Which of the following gets called when an object is being created?

A. Constructor
B. Virtual Function
C. Destructors
D. Main

4. Can we define a class without creating constructors?

A. True
B. False

5. What will be output of following program?

```
#include<iostream>
using namespace std;
class demo{
public:
intf_num,s_num;
sum(inta,int b){
cout<<a+b;
}
};
main(){
demo d1;
d1.sum(d1.f_num=10,d1.s_num=20);
```

return 0;

}

A. 49
B. 50
C. 30
D. 10

6. Which constructor function is designed to copy object of same class type?
A. Copy constructor
B. Create constructor
C. Object constructor
D. Dynamic constructor

7. Allocation of memory to objects at the time of their construction is known as …….. of objects.
A. Run time construction
B. Dynamic construction
C. Initial construction
D. Memory allocator

8. If new operator is used, then the constructor function is
A. Parameterized constructor
B. Copy constructor
C. Dynamic constructor
D. Default constructor

9. Which among the following best describes constructor overloading?
A. Defining one constructor in each class of a program
B. Defining more than one constructor in single class
C. Defining more than one constructor in single class with different signature
D. Defining destructor with each constructor

10. Does constructor overloading include different return types for constructors to be overloaded?
a. Yes, if return types are different, signature becomes different
b. Yes, because return types can differentiate two functions
c. No, return type can't differentiate two functions
d. No, constructors doesn't have any return type

11. Which among the following is possible way to overload constructor?
a. Define default constructor, 1 parameter constructor and 2 parameter constructor
b. Define default constructor, zero argument constructor and 1 parameter constructor
c. Define default constructor, and 2 other parameterized constructors with same signature
d. Define 2 default constructors

12. Which is executed automatically when the control reaches the end of the class scope?

a. Constructor

b. Destructor

c. Overloading

d. Copy constructor

13. The special character related to destructor is ___

a. +

b. !

c. ?

d. ~

14. A destructor is used to destroy the objects that have been created by a ……..

a. Class

b. Object

c. Constructor

d. Destructor

15. …….. Provides the flexibility of using different format of data at runtime depending upon the situation.

a. Dynamic initialization

b. Run time initialization

c. Static initialization

d. Variable initialization

Answers for Self Assessment

1.	D	2.	A	3.	A	4.	A	5.	C
6.	A	7.	B	8.	C	9.	C	10.	D
11.	A	12.	B	13.	D	14.	C	15.	A

Review Questions

1. Write a program to calculate prime number using constructor.
2. Is there any difference between obj x; and objx();? Explain.
3. Can one constructor of a class call another constructor of the same class to initialize the this object? Justify your answers with an example.
4. Should my constructors use "initialization lists" or "assignment"? Discuss.
5. Explain constructor and different types of constructor with suitable example.
6. Write a program that demonstrate working of copy constructor.
7. What about returning a local variable by value? Does the local exist as a separate object, or does it get optimized away?

Further Readings

- E Balagurusamy; Object Oriented Programming with C++; Tata McGraw-Hill.
- Herbert Schildt; The Complete Reference C++; Tata McGraw Hill. Robert Lafore;
- Object-oriented Programming in Turbo C++; Galgotia.

Web Links

- https://en.wikipedia.org/wiki/Object-oriented_programming
- www.web-source.net
- www.webopedia.com